

docs_chain

Kein verfolgtes Dokument kann jemals aus dem Gleichgewicht geraten - inhaltsgehasht, bidirektional, atomar.

Zusammenfassung

Docs Chain ist eine universelle, in Go implementierte bidirektionale Engine zur Abhängigkeitsfortpflanzung zwischen Dokumenten und Datenbanken. Sobald sich ein Element einer registrierten Kette ändert – sei es eine Markdown-Quelle, ein HTML-/PDF-/DOCX-Export oder eine SQLite-Datenbank – erkennt es die Änderung anhand des Inhalts-Hashes und propagiert sie atomar durch alle verbundenen Elemente.

Kurzbeschreibung

Eine Go-Engine, die Dokumente und Datenbanken synchron hält. Durch inhaltsgehashtes, inkrementelles Neurechnen im Salsa-Stil über einen DAG (Kahn-Topologie, frühzeitiger Abbruch, bidirektionale Synchronisationskanten, atomare Umbenennung + SQLite-Transaktions-Commits) werden Exporte neu generiert, sobald sich ein verknüpftes Artefakt ändert.

Ausführliche Beschreibung

Docs Chain ist das, was entsteht, wenn man ein weiteres Mal ein fragiles „Generiere den PDF neu, wenn sich das Markdown ändert“-Shellskript geschrieben hat. Es ersetzt diese ganze Kategorie manuell erstellter Synchronisationslösungen durch eine echte Engine. Es modelliert die Dokumente und Datenbanken eines Projekts als Elemente einer Kette und propagiert bei jeder Änderung eines Elements diese Änderung in alle definierten Richtungen – wobei Exporte atomar neu generiert werden, sodass kein verfolgtes Artefakt jemals aus dem Gleichgewicht geraten kann. Das Design übernimmt seine Präzision direkt aus der Welt inkrementeller Build-Systeme statt aus der Skripting-Welt:

Änderungen werden anhand des **Inhalts-Hashes erkannt, nicht anhand der Änderungszeit**, sodass ein touch nichts auslöst und eine Ein-Byte-Änderung genau die notwendigen Neuberechnungen anstößt – keine Fehlalarme, keine übersehenen Änderungen. Formal ausgedrückt handelt es sich um inhaltsgehashtes, inkrementelles Neurechnen im Salsa-Stil über einen DAG mit Kahn-Topologie, frühzeitigem Abbruch zur Beschneidung unveränderter Teilbäume, deklarierten bidirektionalen sync-Kanten mit Autoritätszuweisung sowie atomarer Umbenennung und SQLite-Transaktions-Commits, sodass ein Absturz während der Propagation niemals einen halb geschriebenen Export hinterlässt. Es wird als vasic-digital-Submodul ausgeliefert und ist integraler Bestandteil des HelixConstitution-Submoduls, sodass jedes Projekt, das die Verfassung übernimmt, Docs Chain standardmäßig erhält und eigene Ketten über kontextspezifische YAML registriert. Die Implementierung ist transparent hinsichtlich des Status (gemäß Verfassung §11.4.6): Phasen 1–4 (Kern-DAG + Hashing, Knotenadapter/-transformationen, Propagations-Orchestrierung mit Atomarität, konfigurationsgesteuerte Multi-Kontext-CLI mit sync/verify/doctor/graph/watch) sind implementiert und getestet; Phase 4b fügt generische bidirektionale md-to-sqlite/sqlite-to-md-Built-ins hinzu (reines Go, zeilenweise Drift-Erkennung, byte-stabile Roundtrips) sowie ein colorize-html-Built-in; Phase 5 (umfassende reale End-to-End-Binärtests) ist implementiert und GRÜN. Phasen 6–7 (Verfassungsverteilung, ATMOSphere-Integration) bleiben GEPLANT und sind operatorengesteuert. Herald ist der erste echte Downstream-Nutzer und synchronisiert ein 66-Dokumente umfassendes Multi-Format-Korpus, das fehlerfrei verifiziert wird.

Warum wir es entwickelt haben

Dokumentation, Exporte und Datenbanken laufen auseinander, sobald sie manuell oder mit fragilen Skripten gepflegt werden. Docs Chain macht die Synchronisation mechanisch, inhaltshashgenau und atomar, sodass eine Änderung an beliebiger Stelle einer Kette alles nachgelagerte (und vorgelagerte) korrekt und sicher aktualisiert.

Warum es ein Game-Changer ist

Es überträgt die hart erkämpften Korrektheitsgarantien, die Compiler- und Build-System-Autoren als selbstverständlich voraussetzen – inhaltsgehashte Abhängigkeitsgraphen, minimale Neuberechnung, atomare Commits – auf Dokumentation und Datenbanken, ein Bereich, der sich historisch mit Cronjobs und gutem Willen begnügt hat. Echte bidirektionale Synchronisation bedeutet, dass die Beziehung zwischen Quelle und Export in beide

Richtungen erzwungen wird, sodass „die Dokumentation ist veraltet“ und „der Export stimmt nicht mit der Quelle überein“ keine wiederkehrenden Fehler mehr sind, sondern Zustände, die das System gar nicht erst zulässt.

Was innovativ ist

- Inkrementelle Neuberechnung auf Basis von Inhalts-Hashes (nicht Änderungszeit) über einen DAG mit vorzeitigem Abbruch.
- Bidirektionale Synchronisationskanten mit deklarierter Autorität (Dokumentation ↔ Exporte ↔ SQLite).
- Atomare Umbenennung + SQLite-Transaktions-Commits für absturzsichere Weitergabe.
- Reiner Go-Rundlauf md-to-sqlite/sqlite-to-md mit Zeilenebenen-Drift-Erkennung.

Herausforderungen & Lösungen

- **Unnötige Neuberechnungen:** gelöst durch Inhalts-Hash-Erkennung statt Zeitstempeln.
- **Teilweise/fehlerhafte Aktualisierungen:** gelöst durch atomare Umbenennung und SQLite-Transaktionen.
- **Korrekte Reihenfolge bei mehreren Mitgliedern:** gelöst durch Kahn-Topologie-Sortierung mit vorzeitigem Abbruch.
- **Ehrliche Fähigkeitsberichterstattung:** gelöst durch Kennzeichnung jeder Phase als IMPLEMENTIERT oder GEPLANT gemäß §11.4.6.

Technologie-Stack (Warum + Wie)

- **Go** – gesamte Engine (internal/hash, graph, adapter, orchestrator, config, state, runner, cmd/docs_chain).
- **DAG + Kahn-Topologie-Sortierung** – Abhängigkeitsreihenfolge mit vorzeitigem Abbruch.
- **SQLite (reines Go modernc)** – Datenbankmitglieder und transaktionale Commits.
- **fsnotify** – watch-Daemon für Live-Weitergabe.
- **YAML-Konfiguration** – Registrierung von Ketten pro Kontext.
- **exec: Transformationen** – Plug-in-basierte Generierung von Markdown→HTML/PDF/DOCX.

Roadmap-Transparenz: Phasen 6–7 (Verteilung der Verfassung, ATMOSphere-Anbindung) sind GEPLANT / operatoren-gesteuert – noch nicht ausgeliefert.